INT8 Post-Training Quantization and Object Size in Aerial Object Detection

Ali Reza Hussaini
Department of Computer Science
Blekinge Institute of Technology (BTH)
Karlskrona, Sweden
alhu22@student.bth.se

Mhd Malek Kisanieh
Department of Computer Science
Blekinge Institute of Technology (BTH)
Karlskrona, Sweden
mhki22@student.bth.se

***Abstract*—INT8 post-training quantization is a common way to make object detectors fast enough for drones, and its accuracy cost is usually reported as one aggregate value. Aerial images are dominated by small objects, which are often assumed to be more sensitive to reduced precision. We tested this assumption in a pre-registered experiment. A YOLOv8n detector was fine-tuned on VisDrone2019-DET and converted with TensorRT into FP32, FP16 and INT8 engines, with three INT8 calibration sets. All engines were evaluated on the same 1,610 test-dev images, and the difference in relative average precision loss between small and non-small objects was estimated with a paired image-level bootstrap. INT8 reduced AP@[.5:.95] by 6.43% (relative) for small objects and by 5.47% for non-small objects. The difference, $D = +0.96$ percentage points, had a 95% confidence interval of $[-0.81, +3.03]$ ($[-0.65, +4.14]$ under a post-hoc cluster bootstrap). Because the interval includes zero, the pre-registered result is inconclusive: the data do not show that INT8 harms small objects more. The difference was positive for all three calibration sets, but no interval excluded zero. The data rule out a size-dependent penalty larger than about four percentage points, which is moderate next to the overall INT8 loss of 5.4%.**

***Index Terms*—aerial imagery, bootstrap, INT8, object detection, post-training quantization, small objects, TensorRT**

I. Introduction

Unmanned aerial vehicles (UAVs) increasingly run object detectors on board, where memory, energy and latency budgets are tight. A standard way to meet these budgets is post-training quantization (PTQ) to 8-bit integers (INT8). A network trained in 32-bit floating point (FP32) is converted without retraining, using a small set of calibration images to estimate the value ranges of its activations [1], [2]. Toolchains such as TensorRT make this conversion a routine export step for YOLO detectors [3], and its accuracy cost is usually reported as a single change in mean average precision (mAP).

A single score can hide uneven effects. In image classification, compressed networks lose accuracy disproportionately on a subset of examples [4]. In aerial detection, the subset of concern is small objects. They make up most annotations in drone benchmarks such as VisDrone [5], [6] and are already the hardest to detect [7]. Two mechanisms make a larger INT8 penalty for small objects plausible. First, a small object is represented by few activations, and their resolution can suffer when large outlier values dominate the quantization range, a known difficulty of PTQ for detectors [2], [8]. Second, a localization error of one pixel lowers the intersection over union (IoU) of a small box much more than that of a large box.

If small objects lose more, aggregate mAP overstates how suitable INT8 is for drones, where small objects such as pedestrians often matter most. If they do not, size-specific countermeasures are not needed for quantization. Detection benchmarks commonly report accuracy by COCO size group [9], but the PTQ studies cited here propose or compare methods using aggregate accuracy and do not test whether the loss depends on object size.

Research question: *In aerial imagery, is the relative loss in detection accuracy caused by INT8 post-training quantization of a lightweight YOLO detector larger for small objects than for non-small objects?*

Variables, hypotheses and prediction. The independent variable is numerical precision: an FP32 baseline versus an INT8 treatment, with an FP16 control. Object size is an observed moderator with two levels, small (box area below 32^2 pixels) and non-small. The dependent variable is COCO AP averaged over IoU thresholds 0.50–0.95 (AP@[.5:.95]) per size group. The checkpoint, input, inference settings, test images, hardware and software are held constant. The test statistic D, defined in Section III, is the relative AP loss of small objects minus that of non-small objects. The hypotheses are $H_0 : D \leq 0$ and $H_1 : D > 0$. If H_1 holds, we predict that the 95% confidence interval (CI) of D lies entirely above zero.

II. Related Work

Quantization. Jacob et al. [1] formalized integer-arithmetic inference with affine quantization of weights and activations. Nagel et al. [2] and Krishnamoorthi [10] review practical PTQ, including per-channel weight quantization and calibration of activation ranges. Wu et al. [11] compare calibration methods for PTQ and find that max calibration, which our toolchain uses, gives inconsistent accuracy, with large drops for networks with outlier activations. They report one aggregate metric per model. INT8 can also be used during training [12], but this changes the trained model and is outside our scope.

Detectors and small objects. Kim et al. [8] propose a new PTQ method that handles outlier activations in object detec-

tors. We instead measure a standard deployment toolchain [3]. The VisDrone benchmark [5] and the VisDrone-DET2021 challenge results [6] show that small, densely packed objects remain difficult, and Akyon et al. [7] address them with slicing-aided inference. Whether object size defines a subset of the kind Hooker et al. [4] found for compressed classifiers is the question studied here.

III. Method

A. Research Design

We ran a quantitative controlled experiment because the question concerns a measurable causal effect of one manipulated factor, precision. A TensorRT FP32 engine is the baseline, an INT8 engine the treatment and an FP16 engine a control. The control is needed because the toolchain runs the non-quantized layers of the INT8 engine in FP16. The design is paired: all conditions come from the same checkpoint and ONNX graph and are evaluated on the same images with the same settings. The design, metrics, size groups and analysis were fixed in a pre-registration dated 7 Oct. 2026, before any model was trained. It is published with the code and a deviation log [13].

For each object-size group g, the relative AP loss is

$$\mathrm{RelDrop}_g = \frac{\mathrm{AP}_g^{\mathrm{FP32}} - \mathrm{AP}_g^{\mathrm{INT8}}}{\mathrm{AP}_g^{\mathrm{FP32}}}. \tag{1}$$

The primary contrast is

$$D = \mathrm{RelDrop}_{\mathrm{small}} - \mathrm{RelDrop}_{\mathrm{non\text{-}small}}. \tag{2}$$

We use the relative loss because small objects start from a much lower AP. The same proportional harm then gives a smaller absolute drop, so an absolute comparison would be biased against H_1.

B. Dataset and Inclusion Criteria

We used VisDrone2019-DET [5], a standard drone benchmark dominated by small objects, with 6,471 training, 548 validation and 1,610 test-dev images. The training split was used for fine-tuning and INT8 calibration, the validation split for checkpoint selection and development, and test-dev only for the final evaluation. The dataset archives were verified by SHA-256.

Annotations are converted to YOLO format for training and to COCO JSON for evaluation. All images of every split are used, without sampling. An annotation is included only if its VisDrone score is one. Score-zero annotations, which are the ignored regions and the "others" category (2,180 and 265 in test-dev), are excluded, so detections on ignored regions count as false positives in every condition. Absolute AP is therefore not directly comparable with the official VisDrone toolkit [6], but the paired comparison is unaffected. Boxes are clipped to the image, and boxes with a width or height below one pixel are discarded (one box, in the training split). Eight automatic integrity checks passed, for example that every image decodes, that the YOLO and COCO labels contain identical boxes and that no image appears in two splits.

Object size is the area of the clipped ground-truth box in original image pixels. Following COCO [9], small objects have an area below $32^2 = 1{,}024$ pixels and non-small objects an area of at least $1{,}024$ pixels. The secondary groups are medium ($32^2 \le \mathrm{area} < 96^2$) and large ($\mathrm{area} \ge 96^2 = 9{,}216$ pixels). Test-dev contains 75,102 objects, of which 67.7% are small, 29.1% medium and 3.2% large (60.5%, 34.0% and 5.5% in the training split). Size and class are related. Pedestrians, people and motors make up 58.5% of small but only 14.4% of non-small objects, and cars, vans, trucks and buses 38.3% and 82.1%, respectively. Size-specific evaluation follows the COCO rule: ground truths outside a size range, detections matched to them, and unmatched detections outside the range are ignored for that range.

C. Procedure

The experiment was run in five steps. (1) A pilot run (three epochs on 10% of the training split, evaluated on validation) tested the whole pipeline. Its results are not used. (2) YOLOv8n [14], the smallest YOLOv8 variant and a typical choice for embedded hardware, was initialized from COCO-pretrained weights and fine-tuned once on the training split at 640×640 for 100 epochs, with batch size 16, random seed 0 and deterministic training. Other options used the Ultralytics defaults, without hyper-parameter search. The best checkpoint on the validation split was frozen. (3) The checkpoint was exported to ONNX with a static input of $1{\times}3{\times}640{\times}640$, and a hash check confirmed that this graph was identical for all conditions. It was converted to FP32, FP16 and INT8 TensorRT engines with the documented Ultralytics workflow [3]. For INT8, NVIDIA ModelOpt inserts explicit quantize/dequantize nodes with max calibration, and unquantized layers run in FP16. Three calibration sets were sampled, one per seed (0, 1 and 2), each with 512 training images drawn uniformly at random. Calibration and evaluation use the same 640×640 letterbox preprocessing. Seed 0 is the primary INT8 condition, and seeds 1 and 2 are robustness checks. (4) All engines were evaluated on the validation split, which was the only split used for development and debugging. (5) Each engine was evaluated on test-dev exactly once (12 Oct. 2026), and the per-image match records were stored for the bootstrap.

D. Measures and Implementation Checks

The PyTorch checkpoint and all engines use the same inference settings: confidence threshold 0.001, non-maximum suppression (NMS) IoU threshold 0.7, at most 500 detections per image, batch size 1, and square 640×640 input. The primary measure is COCO-style AP averaged over IoU thresholds from 0.50 to 0.95 (AP@[.5:.95]), because its strict thresholds also capture the localization errors that motivate the hypothesis. AP at IoU 0.50 (AP@0.5), which mainly reflects whether objects are found, is secondary. Both are reported overall and by size group. Predictions are matched to ground truth with pycocotools, and our AP recomputation must agree with pycocotools to within 10^{-9}. We also compare

the PyTorch FP32 checkpoint with the TensorRT FP32 engine, and our results with the Ultralytics validation metrics.

E. Analysis Plan

AP is computed from detections ranked across the whole test set, so there are no independent per-image AP values for a classical paired test. We therefore use a paired nonparametric bootstrap [15] with the image as the resampling unit. It draws 2,000 resamples of the 1,610 test-dev images with replacement (random seed 0), which gives a Monte Carlo error of about 0.06 percentage points for the interval limits. Every condition uses the same resampled image indices, which keeps the pairing, and AP is recomputed by size group from the stored match records without re-matching.

The confirmatory statistic is D in (2) for FP32 versus INT8 seed 0 on AP@[.5:.95], with a two-sided 95% percentile bootstrap confidence interval. If the lower bound is above zero, H_0 is rejected in favor of a greater relative loss for small objects. If the interval contains zero, the result is inconclusive. If the upper bound is below zero, the result is contrary to H_1. Because H_1 is directional, this rule corresponds to a one-sided test at $\alpha = 0.025$.

The percentile bootstrap makes no normality assumption about D, a ratio-based quantity with an unknown sampling distribution. It does assume that test images are independent and that the resampled AP is well defined. We check that AP is finite in every resample and inspect the bootstrap distribution for skew and bias. The image was chosen as the unit because VisDrone test-dev file names do not reliably identify video sequences. This was decided from the labels alone, before any model output existed. Dependence between images of the same scene is examined with a post-hoc cluster bootstrap.

All other comparisons are exploratory, reported with unadjusted confidence intervals and not used for hypothesis testing: absolute AP drops, AP@0.5, small, medium and large AP, calibration seeds 1 and 2, and the FP16 control. The INT8 result counts as directionally robust if D has the same sign for all three calibration seeds. If FP16 showed a similar size pattern, the effect could not be attributed to INT8 alone.

F. Setup, Deviations and Ethics

All main-study training, engine builds and evaluations ran on one NVIDIA GeForce RTX 5080 (16 GB) in one fixed environment: Python 3.11.9, PyTorch 2.10.0+cu128, Ultralytics 8.4.174 and TensorRT 11.3.0.99. Ultralytics and TensorRT match the pre-registration. Python and PyTorch differ from the versions in the original README (3.12 and 2.14.1), and every run log from the first training onward records the versions actually used. The pre-registered latency benchmark, a secondary outcome, was not implemented, so we report the inference times recorded by Ultralytics, as the pre-registered fallback allows. These deviations (D1, D2), a re-run of the pilot on validation only (D3) and the post-hoc cluster bootstrap (D4) are recorded in a deviation log written after the experiment from the run logs. None of them affects the confirmatory analysis. The study uses only a public research benchmark [5], involves no human participants, and does not redistribute images or use them to identify people.

TABLE I

TEST-DEV AP@[.5:.95] BY OBJECT-SIZE GROUP (0–100 SCALE), WITH 95% PAIRED-BOOTSTRAP CONFIDENCE INTERVALS. NON-SMALL IS THE UNION OF MEDIUM AND LARGE. RELATIVE LOSSES IN THE TEXT ARE COMPUTED FROM UNROUNDED AP VALUES AND CAN DIFFER SLIGHTLY FROM RATIOS OF THE VALUES SHOWN.

Group	Objects	FP32	INT8 seed 0
Small	50,816	6.62 [6.22, 7.19]	6.19 [5.83, 6.72]
Non-small	24,286	25.32 [24.56, 26.49]	23.93 [23.21, 25.05]
Medium	21,876	23.93 [23.19, 25.08]	22.58 [21.87, 23.68]
Large	2,410	35.84 [32.57, 42.52]	33.57 [30.59, 40.63]
All	75,102	15.44 [14.88, 16.15]	14.60 [14.06, 15.28]

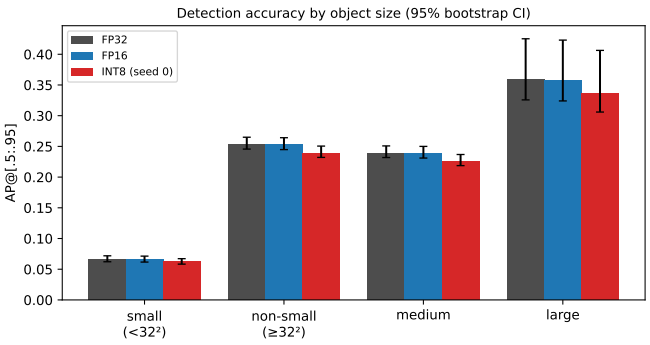


Fig. 1. Test-dev AP@[.5:.95] (0–1 scale) by object-size group for FP32, FP16 and INT8 seed 0. Error bars show 95% paired-bootstrap confidence intervals.

IV. RESULTS

A. Test-Dev Detection Accuracy

Table I and Fig. 1 report AP@[.5:.95] overall and by size group. INT8 seed 0 had lower point estimates than FP32 in every group. In absolute terms, INT8 lost 0.43 AP points for small objects and 1.39 AP points for non-small objects.

B. Confirmatory Test

For INT8 seed 0, the relative AP loss was 6.43% for small objects and 5.47% for non-small objects (Fig. 2). The difference was $D = +0.96$ percentage points, with a 95% confidence interval of $[-0.81, +3.03]$ percentage points. The interval contains zero, so the pre-registered result is inconclusive and H_1 is not supported. Fig. 3 shows the bootstrap distribution of D.

C. Exploratory Comparisons

Table II lists the exploratory contrasts. The point estimate of D was positive for all three calibration seeds ($+0.96$, $+1.83$ and $+1.19$ percentage points), which meets the pre-registered criterion for being directionally robust. All three intervals included zero, however. The overall relative loss varied more between seeds (5.4%, 7.4% and 7.0%) than D did. On AP@0.5, D was close to zero for every seed. The absolute contrast D_{abs} was negative for every seed, with intervals

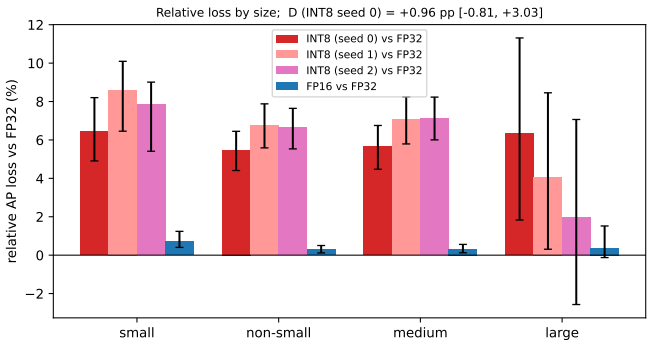


Fig. 2. Relative AP@[.5:.95] loss (%) versus FP32 by object-size group for the three INT8 calibration seeds and FP16. Error bars show 95% paired-bootstrap confidence intervals. The primary comparison is INT8 seed 0 versus FP32.

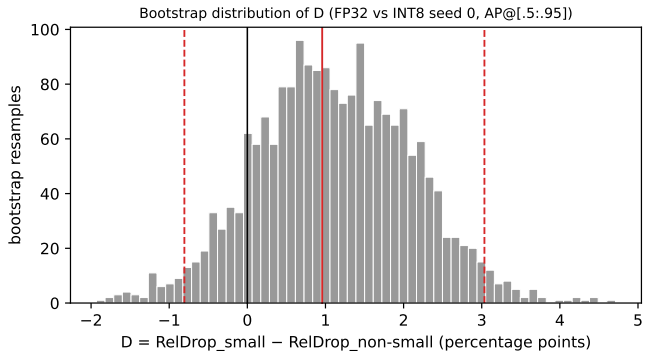


Fig. 3. Paired-bootstrap distribution of D (percentage points) for the primary FP32 versus INT8 seed 0 comparison. The solid line marks the point estimate, the dashed lines the 95% confidence limits, and the zero line no difference in relative loss by size.

TABLE II
RELATIVE AP LOSS VERSUS TENSORRT FP32 (%) FOR SMALL AND NON-SMALL (NON-S.) OBJECTS, THE SIZE CONTRAST D (PERCENTAGE POINTS) AND D_{abs}, THE ABSOLUTE AP DROP FOR SMALL OBJECTS MINUS THAT FOR NON-SMALL OBJECTS (AP POINTS), WITH 95% PAIRED-BOOTSTRAP CONFIDENCE INTERVALS. S0–S2 ARE INT8 CALIBRATION SEEDS 0–2. ALL ROWS USE AP@[.5:.95] EXCEPT THE ROW MARKED AP50, WHICH USES AP@0.5. THE FIRST ROW IS THE CONFIRMATORY TEST, AND ALL OTHER ROWS ARE EXPLORATORY.

Condition	Small	Non-s.	D [95% CI]	D_{abs} [95% CI]
INT8 s0	6.43	5.47	$+0.96$ $[-0.81, +3.03]$	-0.96 $[-1.24, -0.65]$
INT8 s0 (AP50)	4.57	4.87	-0.30 $[-1.74, +1.95]$	-1.42 $[-1.87, -0.79]$
INT8 s1	8.60	6.77	$+1.83$ $[-0.60, +3.72]$	-1.14 $[-1.49, -0.84]$
INT8 s2	7.84	6.65	$+1.19$ $[-1.34, +2.66]$	-1.17 $[-1.49, -0.91]$
FP16	0.70	0.28	$+0.42$ $[+0.07, +0.98]$	-0.02 $[-0.08, +0.03]$

excluding zero. In AP points, small objects lost less than non-small objects.

The FP16 engine lost only 0.37% of overall AP, so TensorRT conversion itself costs little accuracy and the INT8 loss comes mainly from quantization. The FP16 D was $+0.42$ percentage points with an interval just above zero, while its D_{abs} was essentially zero. Because the primary INT8 interval includes zero, the pre-registered FP16 attribution rule is not triggered.

D. Assumption and Implementation Checks

All 2,000 resamples gave finite AP values for every condition and size group. The bootstrap distribution of D (Fig. 3) is unimodal and close to normal (skewness 0.06, excess kurtosis 0.01), with a standard error of 0.99 percentage points, and 12.5% of the resamples had $D \leq 0$. Its mean exceeds the point estimate by 0.16 percentage points. A normal-approximation interval (point estimate ± 1.96 standard errors, $[-0.99, +2.91]$) also contains zero, so the decision does not depend on the interval method. As a post-hoc sensitivity check for dependence between images of the same scene, a cluster bootstrap with 2,000 resamples resampled the 61 filename prefixes, which we treat as possible scenes, instead of single images. It widened the interval of D to $[-0.65, +4.14]$ (standard error 1.20 percentage points) without changing the decision, and the FP16 interval stayed above zero ($[+0.09, +0.90]$).

Our AP matched pycocotools to within 2×10^{-16} for every engine. TensorRT FP32 and PyTorch FP32 differed by less than 10^{-4} AP@[.5:.95] (0–1 scale) in every size group, so the TensorRT FP32 engine is a faithful baseline. Overall AP differed from the Ultralytics validator by 0.0028–0.0029 (0–1 scale) for every TensorRT engine. This offset comes from a different AP implementation and is nearly constant across conditions, so it does not affect the paired contrasts.

E. Latency

The per-image inference times recorded by Ultralytics during test-dev evaluation, an uncontrolled and descriptive measure, were 0.77 ms for FP32, 0.42 ms for FP16 and 0.69–0.71 ms for the three INT8 engines.

V. DISCUSSION

The hypothesis H_1 was not supported. This inconclusive result is not evidence that the two groups are affected equally, but the interval does bound the effect. A size-dependent penalty larger than about 4 percentage points (upper limits $+3.03$ under the image bootstrap and $+4.14$ under the cluster bootstrap) is not compatible with the data. This corresponds to a small-object relative loss above roughly 8.5–9.6% against 5.5% for non-small objects. This bound is moderate next to the overall INT8 loss of 5.4%. Hooker et al. [4] found disproportionate losses on subsets of examples for compressed classifiers, but we could not establish such a subset by object size. For this detector, the aggregate INT8 loss is a reasonable first guide to the loss on small objects, although deployments that depend on small objects should still evaluate them separately.

The relative and absolute views disagree. Small objects lost slightly more in proportion, while non-small objects lost about three times more AP points (1.39 vs. 0.43), so whether quantization "hurts small objects more" depends on the scale. The two mechanisms from the introduction also predict different patterns across IoU thresholds. Coarser activations would tend to make small objects disappear, which should show up at IoU 0.5, while localization jitter would mainly hurt small objects at high IoU thresholds. We found D near zero on AP@0.5 but

slightly positive on AP@[.5:.95]. This is weakly consistent with the localization mechanism, but the difference is well within the uncertainty.

The three calibration seeds share the same model and test images, so their common direction is not an independent replication. The choice of calibration set changed the overall INT8 loss by two percentage points, more than the size contrast itself. This agrees with Wu et al. [11], who found max calibration inconsistent across networks with outlier activations. Outlier-sensitive calibration [8] is a plausible cause of these differences, but our experiment does not test it.

The FP16 D had an interval just above zero under both bootstraps, but its small-object loss is only 0.05 AP points, so this relative difference is fragile. Very small objects may be sensitive to any reduction in precision, or the result may be a chance finding among several uncorrected exploratory intervals.

A. Threats to Validity

Internal validity. Precision is the only manipulated factor. TensorRT does, however, choose kernels and layer fusions per precision, and the INT8 engine runs its unquantized layers in FP16. The FP16 control bounds this toolchain effect but cannot fully separate it from quantization.

Construct validity. Object size is observed, not assigned. Small and non-small objects differ in class composition (Section III), so D measures size and class together. Small objects also start from a low AP of 6.62 AP points, which makes relative losses noisy. The uncontrolled timings show no speed-up of INT8 over FP16 at batch size 1, so the study cannot show that the accuracy loss buys faster inference.

Conclusion validity. The bootstrap captures variation across test images, not from model training or hardware. The cluster bootstrap handles same-scene correlation only coarsely, because five filename prefixes hold 71% of the images. With a CI width of about 4–5 percentage points, the study could detect only a moderate or larger size effect. The pre-registration was not deposited in a time-stamped registry, and the public repository was first committed after the test-dev analysis, so its date rests on our own record.

External validity. We used one YOLOv8n model, one dataset and a 640×640 full-image input, which downscales the original images. Slicing-aided inference [7] enlarges small objects, so the findings should not be generalized to tiled pipelines, larger detectors or other quantization toolchains.

VI. Conclusion

We tested whether INT8 post-training quantization of YOLOv8n causes a greater relative accuracy loss for small than for non-small objects on VisDrone2019-DET. The relative AP loss was 6.43% for small and 5.47% for non-small objects, a difference of 0.96 percentage points (95% CI $[-0.81, +3.03]$), so a greater loss for small objects was not established. The consistent direction across three calibration seeds did not resolve this uncertainty. In absolute AP points, small objects lost less than non-small objects, with exploratory intervals excluding zero. The FP16 control lost almost no accuracy, so the overall loss of 5–7% comes mainly from INT8 quantization, but the size pattern is not necessarily specific to INT8.

Future work should test more models and datasets and measure inference speed under a controlled protocol. Size and class could be separated by estimating D within one class, such as cars, which are numerous in both groups (15,112 small and 12,962 non-small), or by rescaling test images so that the same objects cross the size threshold.

Use of Generative AI

Generative AI tools (Claude by Anthropic and ChatGPT by OpenAI) were used while preparing this article: to review, edit and shorten the text, to check the reported numbers and bootstrap statistics against the result files, to verify the reference details, to implement the post-hoc cluster bootstrap and to draft the deviation log from the run logs. The authors checked all AI output and take full responsibility for the content.

References

[1] B. Jacob *et al.*, "Quantization and training of neural networks for efficient integer-arithmetic-only inference," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, 2018, pp. 2704–2713, doi: 10.1109/CVPR.2018.00286.

[2] M. Nagel, M. Fournarakis, R. A. Amjad, Y. Bondarenko, M. van Baalen, and T. Blankevoort, "A white paper on neural network quantization," arXiv:2106.08295, 2021.

[3] Ultralytics, "TensorRT export for YOLO26 models," *Ultralytics Documentation*. [Online]. Available: https://docs.ultralytics.com/integrations/tensorrt. Accessed: Oct. 22, 2026.

[4] S. Hooker, A. Courville, G. Clark, Y. Dauphin, and A. Frome, "What do compressed deep neural networks forget?" arXiv:1911.05248, 2019.

[5] P. Zhu *et al.*, "Detection and tracking meet drones challenge," *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 44, no. 11, pp. 7380–7399, Nov. 2022, doi: 10.1109/TPAMI.2021.3119563.

[6] Y. Cao *et al.*, "VisDrone-DET2021: The vision meets drone object detection challenge results," in *Proc. IEEE/CVF Int. Conf. Comput. Vis. Workshops (ICCVW)*, 2021, pp. 2847–2854, doi: 10.1109/ICCVW54120.2021.00319.

[7] F. C. Akyon, S. O. Altinuc, and A. Temizel, "Slicing aided hyper inference and fine-tuning for small object detection," in *Proc. IEEE Int. Conf. Image Process. (ICIP)*, 2022, pp. 966–970, doi: 10.1109/ICIP46576.2022.9897990.

[8] M. Kim, D. Lee, J. Yu, J. Hur, G. Kim, and J. Kim, "Inlier-centric post-training quantization for object detection models," arXiv:2602.03472, Feb. 2026.

[9] T.-Y. Lin *et al.*, "Microsoft COCO: Common objects in context," in *Proc. Eur. Conf. Comput. Vis. (ECCV)*, 2014, pp. 740–755, doi: 10.1007/978-3-319-10602-1_48.

[10] R. Krishnamoorthi, "Quantizing deep convolutional networks for efficient inference: A whitepaper," arXiv:1806.08342, 2018.

[11] H. Wu, P. Judd, X. Zhang, M. Isaev, and P. Micikevicius, "Integer quantization for deep learning inference: Principles and empirical evaluation," arXiv:2004.09602, 2020.

[12] F. Zhu *et al.*, "Towards unified INT8 training for convolutional neural network," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, 2020, pp. 1966–1976, doi: 10.1109/CVPR42600.2020.00204.

[13] A. R. Hussaini and M. M. Kisanieh, "int8_size_study: Code, pre-registration and results," GitHub repository, 2026. [Online]. Available: https://github.com/Malek-Kisanieh/int8_size_study. Accessed: Oct. 22, 2026.

[14] G. Jocher, A. Chaurasia, and J. Qiu, "Ultralytics YOLOv8," version 8.0.0, software, 2023. [Online]. Available: https://github.com/ultralytics/ultralytics. Accessed: Oct. 22, 2026.

[15] B. Efron and R. J. Tibshirani, *An Introduction to the Bootstrap*. New York, NY, USA: Chapman & Hall, 1993.